CLAUDE CODE · INVENTARIO LOCAL

Manual de skills instaladas

Estado completo de `~/.claude/skills/` y `~/.claude/plugins/` en esta máquina: dos paquetes documentados a fondo (instalación, configuración, uso) y el resto del catálogo en un índice de referencia.

69	1	1	1	1	7
SKILLS TOTALES	SECURITY-AUDIT	BRAG	DIAGRAM-DESIGN	ANATOMY	UI-UX-PRO-MAX

1	58
PLUGIN (CLAUDE-MEM)	SUITE GSTACK + OTRAS

security-audit · ui-ux-pro-max · claude-mem · brag · diagram-design · anatomy · Inventario completo · Referencia: ECC · Gestión de skills

- **Estado verificado el 9 oct 2026 en esta Mac:** instaladas `security-audit`, `brag`, `diagram-design`, `anatomy`, las 7 skills de ui-ux-pro-max, la suite `gstack` (v1.91.68, vía `./setup` con Bun 1.4.2) y el plugin `claude-mem` (v13.35.0). Abre una sesión nueva para que carguen.

security-audit

1 SKILL · CLOUDFLARE

`~/.claude/skills/security-audit/`

QUÉ HACE

Convierte al agente en auditor de seguridad. Orquesta agentes aislados a lo largo de seis fases: reconocimiento, caza guiada por cobertura, validación de candidatos, salida estructurada (`findings.json`), verificación independiente de los registros y reporte neutral (`REPORT.md`, `FINDINGS-DETAIL.md`, `NEEDS-VALIDATION.md`). Es la skill que dio origen al arnés de descubrimiento de vulnerabilidades de Cloudflare.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/cloudflare/security-audit-skill

# copiado skills/security-audit/ a
~/.claude/skills/security-audit/

# alternativa oficial (Skills CLI) no usada aquí
npx skills add https://github.com/cloudflare/security-audit-skill \
  --skill security-audit --global
```

↗ Repositorio en GitHub ↗ Blog de Cloudflare

CÓMO SE USA

- Pedidos como `security audit this codebase`, `find security vulnerabilities in ./src` o `do a security review, output to ~/audits/mi-proyecto`.
- **Modo guía** (por defecto): preguntas de seguridad o revisiones puntuales, sin crear archivos.
- **Modo auditoría completa**: solo si pides auditar o hacer pen-test de un repo, o reportes. Salida por defecto en `~/security-audit-skill/<repo>/run-N`.
- Varias ejecuciones sobre el mismo repo son aditivas: una sola corrida encontró aprox. la mitad de lo que encontraron las repetidas.

REQUISITOS

Node.js para los validadores (`validate-findings.cjs`, `validate-coverage-ledger.cjs`; sin dependencias). Presente aquí: `v22.23.2`. Para ejecutar código del objetivo exige un sandbox del SO sin red; sin él, deja el hallazgo como `needs_validation` en vez de ejecutar nada.

ui-ux-pro-max

`~/.claude/skills/{ui-ux-pro-max, design, design-system, ui-styling, brand, banner-design, slides}/`

7 SKILLS

QUÉ HACE

Inteligencia de diseño UI/UX searchable en local: 79 estilos visuales (50 activos), 192 paletas de color con perfil de razonamiento por tipo de producto, 74 combinaciones tipográficas, 119

guías de UX, 105 iconos curados, 17 presets de GSAP, 25 tipos de gráfico y 22 stacks técnicos (React, Next.js, Vue, Nuxt, Svelte, Astro, SwiftUI, Flutter, Tailwind, shadcn/ui, Jetpack Compose, Angular, Laravel, Three.js, y más).

El plugin trae siete skills relacionadas, no solo una. El repo publica el paquete completo bajo `.claude/skills/`, así que instalar "el skill" instala las siete.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/nextlevelbuilder/ui-ux-pro-max-skill.git

# copiados los 7 directorios de .claude/skills/ del repo a
~/.claude/skills/ui-ux-pro-max/
~/.claude/skills/design/
~/.claude/skills/design-system/
~/.claude/skills/ui-styling/
~/.claude/skills/brand/
~/.claude/skills/banner-design/
~/.claude/skills/slides/

# alternativas oficiales no usadas aquí
/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill
/plugin install ui-ux-pro-max@ui-ux-pro-max-skill

npx ui-ux-pro-max-cli init --ai claude --global
```

↗ Repositorio en GitHub ↗ uupm.cc

- Se copiaron los archivos estáticos directo del repo en vez de correr `npx ui-ux-pro-max-cli` — evita ejecutar el instalador npm de un tercero; el resultado en disco es el mismo.

LAS 7 SKILLS DEL PAQUETE

ui-ux-pro-max
Motor principal: búsqueda de estilos, paletas, tipografía, charts y reglas UX/accesibilidad por prioridad.

design
Identidad de marca, tokens, logos (55 estilos), CIP, banners, iconos, fotos sociales.

design-system
Arquitectura de tokens en tres capas (primitivo→semántico→componente) y specs de componentes.

ui-styling
Componentes accesibles con shadcn/ui (Radix +

brand
Voz de marca, mensajería, gestión de

banner-design
Banners para redes, ads y hero de sitio — 22 estilos, múltiples

Tailwind) y diseños basados en canvas.

assets y consistencia de guías de estilo.

formatos.

slides

Presentaciones HTML con Chart.js, tokens de diseño y fórmulas de copywriting.

CÓMO SE DISPARA

- → Automático al diseñar, construir o revisar cualquier UI: páginas, componentes, paletas, tipografía, layout, accesibilidad, animación.
- → También responde a pedidos explícitos de logo, banner, CIP, icono o presentación HTML.
- → Se salta en lógica de backend, APIs, infraestructura o scripts sin impacto visual.

REQUISITO

Python 3.x para el script de búsqueda (solo librería estándar, no instala nada ni hace llamadas de red). Ya está presente en esta máquina: `Python 3.14.6`.

claude-mem

`~/.claude/plugins/marketplaces/thedotmack/ · cache en .../cache/thedotmack/claude-mem/13.35.0`

PLUGIN + WORKER

QUÉ HACE

Sistema de memoria persistente para Claude Code. Captura observaciones de uso de herramientas durante la sesión, genera resúmenes semánticos y los deja disponibles para sesiones futuras del mismo proyecto — sin que el usuario tenga que repetir contexto.

A diferencia de ui-ux-pro-max, no es un conjunto de archivos estáticos: registra hooks de ciclo de vida en Claude Code y corre un worker HTTP en segundo plano.

INSTALACIÓN Y CONFIGURACIÓN (YA APLICADA, 9 OCT 2026)

```
# instalado sin cuenta / sin nube, con el plan de Anthropic ya logueado
CLAUDE_MEM_ONLINE_OPTIN=false npx claude-mem@latest install \
  --ide claude-code --provider claude
```

```
# telemetría anónima apagada
npx claude-mem telemetry disable

# worker local iniciado manualmente (el installer lo dejó sin arrancar)
npx claude-mem start
```

↗ Repositorio en GitHub ↗ Web viewer del worker

ESTADO ACTUAL

CONFIG	VALOR
Proveedor	`claude` — usa la sesión de Anthropic ya logueada, sin API key aparte
Cloud sync	OFF — nada se manda a cmem.ai salvo el contacto único de alta si te logueas
Telemetría	OFF — deshabilitada explícitamente tras instalar
Worker	Corriendo en `http://127.0.0.1:37701`
Datos	Todo local en `~/.claude-mem/`

- **Resuelto el 9 oct 2026:** se instaló el CLI independiente (`npm install -g @anthropic-ai/claude-code` , v2.1.296) y se hizo `claude auth login` con la cuenta de Claude (plan Pro). Tras reiniciar el worker (`npx claude-mem restart`) ya no aparece *degraded (Claude CLI setup required)*. La memoria se acumula al usar Claude Code en un proyecto y se inyecta desde la segunda sesión de ese proyecto.

SKILL PRINCIPAL + COMANDOS

El plugin expone `mem-search` como la skill de cara al usuario (búsqueda de observaciones pasadas), más ~19 skills internas de soporte (`learn-codebase` , `how-it-works` , `knowledge-agent` , `standup` , `agent-cost-report` , `weekly-digests` , entre otras) para flujos específicos del propio proyecto claude-mem.

COMANDO	QUÉ HACE
`npx claude-mem status`	Estado del worker (PID, puerto, uptime, dependencias)
`npx claude-mem doctor`	Diagnóstico de runtime (Bun, uv, worker, plugin)

`npx claude-mem start / stop / restart`	Control manual del worker en segundo plano
`npx claude-mem search "<query>"`	Busca observaciones guardadas desde la terminal
`npx claude-mem telemetry status`	Confirma que la telemetría sigue apagada
`npx claude-mem uninstall`	Quita el plugin, hooks y configuración

brag

1 SKILL · VIDEO

~/.claude/skills/brag/

QUÉ HACE

Convierte el proyecto actual en un video corto de lanzamiento, con música, movimiento y copy para compartir. Lee el código del proyecto directamente, sin necesitar URL en vivo ni capturas de pantalla. La composición del video la ejecuta Hyperframes; la skill decide el ángulo de producto, el tono y qué momentos mostrar.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/latent-spaces/brag.git

# copiado skills/brag/ a
~/.claude/skills/brag/
  ├── SKILL.md
  ├── assets/    # música y efectos de sonido incluidos
  ├── references/
  └── scripts/

# alternativa con el instalador oficial (cualquier agente)
npx skills add https://github.com/latent-spaces/brag --skill brag

# o como plugin nativo de Claude Code
/plugin marketplace add latent-spaces/brag
/plugin install brag@brag
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Hyperframes

REQUISITOS PARA EJECUTARLA

- → Node.js 22+
- → FFmpeg disponible en `PATH`
- → CLI de Hyperframes; verifícalo con `npx hyperframes doctor`

USO BÁSICO

1. Párate en la carpeta del proyecto que quieres mostrar.
2. Escribe `let's /brag` o simplemente `/brag`.
3. La skill lee el código y arma un guion y un storyboard cortos.
4. Entrega un brief de composición a Hyperframes, que renderiza el video.
5. Recibes una carpeta `brag-output/` con plan, brief, copy para redes y `brag.mp4`.

OPCIONES (FLAGS)

FLAG	VALORES	POR DEFECTO
`--tone`	preset o descripción libre	inferido
`--format`	`landscape` / `vertical` / `square`	`landscape`
`--duration`	segundos	auto (15–25 s)
`--no-music`	flag	música activada
`--no-sfx`	flag	efectos activados
`--title`	texto	inferido del proyecto
`--voice`	flag	narración apagada

Los presets de tono son `default`, `polished`, `yc-parody`, `chaotic`, `deadpan`, `cinematic` y `app-store`. También acepta dirección libre:

```
/brag --tone "fake Series A launch from 2016"
/brag --voice
```

- La narración usa Kokoro vía Hyperframes y solo se activa con `--voice`. Los requisitos (FFmpeg, Hyperframes) no se han verificado en esta máquina.

diagram-design

1 SKILL · 45 TIPOS

~/.claude/skills/diagram-design/

QUÉ HACE

Dibuja diagramas como un único archivo HTML con SVG incrustado, con estilo editorial y sin sombras. Cubre 45 tipos: arquitectura, flujo, secuencia, máquina de estados, ER, línea de tiempo, swimlane, cuadrante, radar, Sankey, Gantt, mapa de Wardley, kanban, UML, esquema de base de datos y más. El agente elige el tipo, explica su plan y escribe el archivo, que se abre con doble clic.

INSTALACIÓN (YA APLICADA)

```
# clonado desde
git clone https://github.com/cathrynlavery/diagram-design.git

# copiado a
~/.claude/skills/diagram-design/
  ├── SKILL.md
  ├── assets/
  ├── references/
  └── scripts/   # importadores y exportador, solo Python local

# alternativa con el instalador oficial (cualquier agente)
npx skills add cathrynlavery/diagram-design

# o como plugin de Claude Code (añade comandos slash)
/plugin marketplace add cathrynlavery/diagram-design
/plugin install diagram-design@diagram-design
```

↗ Repositorio en GitHub ↗ Ver en skills.sh ↗ Sitio web ↗ Galería

USO BÁSICO

1. Pide el diagrama en lenguaje natural, por ejemplo `Hazme un diagrama de arquitectura: frontend, backend, base de datos y caché Redis`.
2. La primera vez en un proyecto, pregunta si quieres adaptar el estilo a tu marca: URL, skill instalada, carpeta, tokens pegados, estilo por defecto o perfil guardado.
3. El agente indica qué tipo de diagrama va a usar y escribe un archivo `.html`.
4. Abre el archivo en el navegador, o pide exportarlo a PNG o SVG.

IMPORTAR Y EXPORTAR

- → Importa archivos `.drawio`, Mermaid y `.excalidraw` y los redibuja en tu estilo.
- → Exporta a PNG o SVG con `/export-diagram` (versión plugin).
- → `/doctor` revisa que la instalación esté completa (versión plugin).

- Sin servidor, cuentas ni telemetría. Lo único que sale a internet son las fuentes de Google Fonts cuando abres un diagrama en el navegador.

anatomy

1 SKILL · FIGURAS

`~/.claude/skills/anatomy/`

QUÉ HACE

Explica una idea técnica construyéndola como un objeto físico que funciona. Una clave de caché que evita trabajo repetido se vuelve una cerradura de pines que solo gira si todos los pines coinciden. Genera figuras isométricas interactivas en SVG, con muchas piezas pequeñas sombreadas, dentro de una tarjeta con título, pista y lectura en vivo. Puede añadir shaders WebGL (fuego, agua, luz) dentro del mismo dibujo.

Cada número que muestra la figura sale del mismo modelo que mueve sus piezas. Funciona sin framework (TypeScript/JS que escribe SVG y HTML) y en React/Next.js.

INSTALACIÓN (YA APLICADA)

```
# el repositorio completo es la carpeta de la skill
git clone https://github.com/wheresryan22/anatomy.git
# copiado (sin .git) a
~/.claude/skills/anatomy/

# alternativa con el instalador oficial
npx skills add wheresryan22/anatomy -g -a claude-code

# otros agentes: cambia -a por opencode o codex
```

↗ Repositorio en GitHub ↗ Sitio web ↗ Descargar ZIP

REQUISITOS

- → Node.js 20.10 o superior. Presente aquí: `v24.19.0`.

- → Chrome o Chromium para las comprobaciones de la figura. No verificado en esta máquina.
- → Sin API keys ni `npm install`: usa solo módulos integrados de Node.

USO

```
/anatomy explain how a token bucket rate limiter works, interactive
```

- → También se activa solo ante pedidos de ilustraciones isométricas, figuras explicativas para docs o blogs, o SVG que muestren cómo funciona un sistema.
- → El repo incluye 8 ejemplos en `examples/` (motor Raptor, tanque de ondas, indicador de dial, máquina arcade y otros).
- → Puede publicar una figura terminada en la galería comunitaria, solo si se lo pides.

- Los scripts de captura solo lanzan Chrome local y se comunican con `127.0.0.1`. No hay llamadas a servicios externos en el código de la skill.

Inventario completo

Resto de ~/.claude/skills/ — presentes desde antes de esta sesión (agosto 2026)

58 SKILLS · RESUMEN

Suite **gstack** (automatización de todo el ciclo: diseño, QA, ship, revisiones) más un puñado de skills sueltas que viven en la misma carpeta de usuario. No se documentan a nivel de instalación porque ya estaban en la máquina; aquí solo el resumen de qué hace cada una.

SKILL	QUÉ HACE
`_gstack-command`	Router interno de la suite gstack.
`gstack`	Router principal de la suite gstack.
`gstack-upgrade`	Actualiza gstack a la última versión.
`autoplan`	Pipeline de auto-revisión: corre CEO, diseño, ingeniería y DX en secuencia con decisiones automáticas.
`plan-ceo-review`	Revisión de plan en modo CEO/fundador.
`plan-design-review`	Revisión de plan con ojo de diseñador, interactiva.

`plan-devex-review`	Revisión interactiva de developer experience sobre un plan.
`plan-eng-review`	Revisión de plan en modo eng manager.
`plan-tune`	Auto-ajuste de sensibilidad de preguntas y perfil psicográfico del developer.
`design-consultation`	Propone un sistema de diseño completo (estética, tipografía, color, layout, motion) con previews.
`design-html`	Finalización de diseño: genera HTML/CSS de producción.
`design-review`	QA de diseño: inconsistencias visuales, jerarquía, patrones de "AI slop", interacciones lentas.
`design-shotgun`	Genera varias variantes de diseño con IA, arma comparación y recoge feedback.
`devex-review`	Auditoría en vivo de developer experience.
`diagram`	Convierte una descripción en inglés (o mermaid) en diagrama + .excalidraw editable.
`document-generate`	Genera documentación faltante desde cero para una feature o proyecto.
`document-release`	Actualización de documentación post-lanzamiento.
`health`	Dashboard de calidad de código.
`investigate`	Debugging sistemático con investigación de causa raíz.
`qa`	QA sistemático de una app web, con fixes de los bugs encontrados.
`qa-only`	QA de solo reporte, sin aplicar fixes.
`review`	Revisión de PR previa a mergear.
`retro`	Retrospectiva semanal de ingeniería.
`land-and-deploy`	Flujo de land + deploy.
`landing-report`	Dashboard de solo lectura para la cola de ship por workspace.

`ship`	Flujo de ship completo: merge de base, tests, revisión de diff, bump de versión, changelog, commit, push, PR.
`setup-deploy`	Configura los ajustes de deploy para `/land-and-deploy`.
`canary`	Monitoreo canary post-deploy.
`benchmark`	Detección de regresiones de rendimiento usando el daemon de browse.
`benchmark-models`	Benchmark cruzado entre modelos para las skills de gstack.
`browse`	Navegador headless rápido para QA y dogfooding de sitios.
`setup-browser-cookies`	Importa cookies de tu Chromium real a la sesión headless de browse.
`open-gstack-browser`	Lanza GStack Browser — Chromium controlado por IA con extensión de sidebar.
`pair-agent`	Empareja un agente de IA remoto con tu navegador.
`scrape`	Extrae datos de una página web.
`skillify`	Convierte el último `/scrape` exitoso en una skill de navegador permanente.
`codex`	Wrapper del CLI de OpenAI Codex — tres modos.
`context-save`	Guarda el contexto de trabajo actual.
`context-restore`	Restaura el contexto guardado por `/context-save`.
`learn`	Gestiona los aprendizajes del proyecto.
`spec`	Convierte una intención vaga en un spec preciso y ejecutable, en cinco fases.
`make-pdf`	Convierte cualquier markdown en un PDF de calidad de publicación.
`office-hours`	YC Office Hours — dos modos.
`freeze`	Restringe la edición de archivos a un directorio específico durante la sesión.

`connect-chrome`	Conecta el navegador de gstack con tu Chrome.
`test-audit`	Auditoría de la calidad y cobertura de los tests.
`deslop-shared-libs`	Limpia código genérico ("slop") en librerías compartidas.
`unfreeze`	Quita el límite puesto por `/freeze`.
`guard`	Modo de seguridad completo: avisos de comandos destructivos + edición limitada por directorio.
`careful`	Barreras de seguridad para comandos destructivos.
`cso`	Modo Chief Security Officer.
`security-gap-checklist`	Checklist fijo de 10 puntos de seguridad (rate limiting, secretos, RLS, validación, etc.), afinado para Supabase/Postgres.
`setup-gbrain`	Configura gbrain: instala el CLI, inicializa un brain local o en Supabase, registra el MCP.
`sync-gbrain`	Mantiene gbrain sincronizado con el código del repo.
`ios-clean`	Quita el paquete DebugBridge SPM y el cableado `#if DEBUG` de una app iOS.
`ios-design-review`	Auditoría de diseño visual para apps iOS en hardware real.
`ios-fix`	Corrector autónomo de bugs iOS.
`ios-qa`	QA en vivo sobre dispositivo para apps SwiftUI.
`ios-sync`	Regenera el debug bridge de iOS contra las plantillas gstack más recientes.

ECC (Everything Claude Code)

SOLO REFERENCIA

Referencia · no instalado · paquete npm `ecc-universal` v2.2.3

QUÉ ES

Un "sistema operativo de agente" para Claude Code (con adaptadores limitados para Codex, Cursor, OpenCode, Gemini y otros). No es una sola skill: es un kit de agentes, skills, hooks y reglas con licencia MIT que impone un ciclo de ingeniería fijo en lugar de improvisar en cada prompt.

```
plan → test → implement → review → verify → remember → improve
```

QUÉ INCLUYE

71 agentes
Planificación, revisión por lenguaje, reparación de builds, seguridad, arquitectura, base de datos, E2E.

306 skills
TDD, investigación, seguridad, docs, frontend, datos, ML, operaciones, patrones de agentes.

97 comandos
Atajos heredados mientras el proyecto migra a un modelo basado en skills.

Hooks y memoria
Se disparan antes y después de cada herramienta: bloquean o avisan, guardan resúmenes de sesión y aprenden patrones ("instincts").

Reglas
Estándares siempre cargados que eliges por lenguaje o proyecto (Python, Go, Rust, React, Swift...).

AgentShield
Escáner de seguridad para prompts, hooks, MCP, permisos y secretos de tus archivos de agente.

INSTALACIÓN (SI DECIDES USARLO)

```
# guiado; requiere Node 18+, Git y Claude Code 2.1+
npx ecc-universal@2.2.3 setup

# o como plugin de Claude Code (elige un solo método, no los apiles)
ecc@ecc
```

↗ Repositorio en GitHub ↗ ecc.tools

- **Antes de instalar:** solapa con `claude-mem` (memoria), `gstack` (revisión, QA y envío) y `security-audit` , así que habría varios sistemas de hooks y memoria compitiendo, además de más contexto cargado en cada sesión. Los hooks ejecutan código local en cada acción: revisa el perfil de hooks antes de activarlo. Es un proyecto de un solo mantenedor; el núcleo es gratis y hay una versión de pago (ECC Pro, GitHub App para repos privados).

GESTIÓN DE ESTAS SKILLS

- Todas se activan solas cuando el pedido coincide con su descripción, o al invocarlas con `/nombre-skill`.
- ui-ux-pro-max y la suite gstack están instaladas a nivel de usuario (`~/.claude/skills/`): aplican a cualquier proyecto que abras en esta máquina. Para quitar una, borra su carpeta; para actualizarla, vuelve a clonar y sobrescribe.
- claude-mem es distinto: es un **plugin** gestionado por Claude Code (`~/.claude/plugins/`), con un worker en segundo plano. Se administra con `npx claude-mem <comando>` o `/plugin uninstall claude-mem`, no borrando carpetas a mano.
- Este documento refleja el estado al momento de generarlo — si instalas o quitas algo después, pide que se vuelva a actualizar.

Generado y actualizado desde Claude Code en esta máquina

69 skills · security-audit (1) + brag (1) + diagram-design (1) + anatomy (1) + ui-ux-pro-max (7) + gstack y otras (58) + claude-mem (plugin)